



PreDyn-IDS: Pretrained lightweight open-set IDS for IoT via dynamic confidence and hierarchical feature fusion

Xi Luo^a, Guohua Yan^a, Lihua Yin^{a,*}, Gaopan Hou^b, Jingjing Guo^c, Yanqi Song^d, Xiang Li^e, Wenting Wang^f, Tianqing Zhu^g

^a Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, 511370, China

^b Hangzhou Institute of Technology, Xidian University, Hangzhou, 311231, China

^c School of Cyber Engineering, Xidian University, Xi'an, 710071, China

^d Security Research Department, China Academy of Information and Communications Technology, Beijing, China

^e College of Cryptology and Cyber Science, Nankai University, Tianjin, 300350, China

^f State Grid Shandong Electric Power Research Institute, Jinan, 250003, China

^g The Faculty of Data Science, City University of Macau, Macau, China

ARTICLE INFO

Keywords:

Feature fusion
IoT network
Intrusion detection system
Self-supervised pretraining
Open-set classification

ABSTRACT

In IoT environments, intrusion detection systems are critical for securing networked control systems. Recent approaches leverage self-supervised pretraining to reduce reliance on labeled data, yet most still depend on computationally expensive sequential models and operate under closed-set assumptions, limiting their effectiveness against previously unseen attacks. In this paper, we propose PreDyn-IDS, a lightweight open-set intrusion detection framework for IoT that is both pretrained and confidence-aware. It leverages self-supervised pretraining to enable cross-granularity feature fusion between a WAC module (for packet-level frequency) and Mamba (for flow-level context), achieving efficient feature extraction, and employs a dynamic confidence mechanism to classify known attacks and separate unknown traffic during both fine-tuning and detection. Experimental results on the IoT dataset (Edge-IoT) and general network traffic datasets (ISCXVPN, ISCXTor) show that PreDyn-IDS outperforms state-of-the-art methods in both detection accuracy and unknown threat identification, demonstrating its effectiveness and practicality in real-world IoT scenarios.

1. Introduction

The convergence of the Internet of Things (IoT) and cyber-physical systems has enabled real-time monitoring and remote control across a wide range of applications. However, this tight integration of digital and physical components significantly expands the attack surface, rendering IoT networks vulnerable to cyber threats such as malware, denial-of-service (DoS) attacks, and protocol-specific exploits. There is a growing need for specialized intrusion detection systems that can effectively identify malicious activities in resource-constrained and protocol-heterogeneous settings. Network-based intrusion detection systems (NIDS) play a critical role in securing Internet of Things environments by monitoring network traffic in real time. Due to the resource constraints and protocol specificity of many IoT devices, traditional host-based solutions are often impractical. NIDS offer a non-intrusive alternative, capable of detecting anomalies and cyber threats—such as

spoofing, denial-of-service, and protocol violations—by analyzing communication patterns across the network [1,2].

Recent advancements leverage deep learning to model normal behavior and identify deviations, enabling effective threat detection without requiring modifications to legacy IoT devices. However, current intrusion detection systems in the IoT domain face significant challenges. First, anomaly detection is typically performed on resource-constrained IoT gateways, whose limited computing capacity and memory impose strict constraints on the complexity and inference efficiency of detection models. Second, labeled attack datasets are scarce and often lack diversity, hindering the training and evaluation of data-hungry machine learning approaches, publicly available IoT datasets such as TON_IoT [3] and Edge-IoTset [4], include only a limited set of attack types (e.g., replay or unauthorized write commands). Third, most existing systems are designed to detect known attack patterns and struggle to identify zero-day or previously unseen threats, limiting their adaptability in dynamic

* Corresponding author.

E-mail addresses: xlue@gzhu.edu.cn (X. Luo), Guohuayan@e.gzhu.edu.cn (G. Yan), yinlh@gzhu.edu.cn (L. Yin), hogaopan@xidian.edu.cn (G. Hou), jjguo@xidian.edu.cn (J. Guo), songyanqi@caict.ac.cn (Y. Song), lixiang@nankai.edu.cn (X. Li), 12232093@zju.edu.cn (W. Wang), tqzhu@cityu.edu.mo (T. Zhu).

<https://doi.org/10.1016/j.inffus.2026.104611>

Received 2 February 2026; Received in revised form 13 May 2026; Accepted 8 July 2026

Available online 12 July 2026

1566-2535/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

environments. These limitations collectively impede the deployment of robust and generalizable intrusion detection solutions in real-world IoT scenarios.

Pretraining-finetuning strategies have been widely adopted in network intrusion detection due to their effectiveness in mitigating the scarcity of labeled data. In contrast to conventional supervised approaches [5–13] that rely entirely on labeled datasets, recent methods based on pretrained models [14–18] adopt a two-stage paradigm. During the pretraining phase, the model learns deep, general-purpose representations from large volumes of unlabeled traffic through self-supervision. In the subsequent fine-tuning phase, these learned representations are adapted to specific downstream tasks using limited labeled data under supervised learning. But most of the Pretraining methods adopt a single-granularity modeling paradigm, treating network flows directly as raw byte sequences and failing to fuse representations across packet- and flow-level granularities. Feeding large volumes of bytes into large sequential models not only incurs high computational cost but also overlooks the fine-grained structural information embedded in packet-level protocol fields. Consequently, critical semantic details in network protocols—such as command types and function codes are lost, reducing the model’s sensitivity to subtle anomalous behaviors and failing to meet the high-precision detection requirements of IoT. On the other hand, to detect unknown attacks, Several methods aim to detect unknown classes by integrating extreme value theory (EVT) or applying confidence calibration techniques [19–22], but their approach is limited by its dependence on a fixed threshold and strong reliance on historical data, which can lead to high-risk false negatives when facing evolving or zero-day attacks in dynamic network environments.

To address the limitations of the aforementioned methods, we propose a framework named PreDyn-IDS, which integrates pretraining, lightweight modeling, and anomaly-aware classification to achieve efficient and robust threat detection. Our approach leverages a self-supervised pretraining strategy on unlabeled network traffic, enabling effective feature learning without relying on large-scale labeled attack datasets, alleviating the data scarcity problem. Second, we adopt the Mamba [23] architecture as the flow-level context learning module, which is inherently lightweight and highly efficient, significantly reducing computational overhead while maintaining strong sequential modeling capability, making it suitable for resource-constrained IoT environments. Third, our framework incorporates a Confidence Mechanism of Dynamic Threshold (CMDT) mechanism that dynamically adjusts detection thresholds based on model confidence, allowing it to identify unknown attacks by detecting deviations from learned normal behavior. By combining pretraining for data efficiency and adaptive thresholding for unknown threat detection, our method provides a low-resource and generalizable solution for real-world IoT security.

Our contributions can be summarized as follows:

- We propose a lightweight self-supervised pretraining framework that eliminates the need for large-scale labeled data, effectively addressing data scarcity in IoT scenarios. By fusing fine-grained packet-level spatial-frequency features with coarse-grained flow-level contextual features, the framework achieves cross-granularity features fusion, significantly improving feature learning efficiency for IoT network traffic.
- We design the CMDT, simultaneously optimizing classification accuracy for known categories and detecting unknown attack classes. This approach avoids the forced misclassification issues inherent in traditional closed-set classification methods, enabling our framework to be effectively deployed in IoT environments.
- We validate the framework’s efficiency and robustness on IoT traffic datasets (Edge-IIOT) and conduct fair compar-

isons on standard non-IoT benchmarks (e.g., ISCX-IOT, ISCX-VPN). Results show our method significantly outperforms existing approaches in detection accuracy, computational efficiency, and generalization capability, proving its adaptability to future complex IoT informatization scenarios.

2. Related work

We organize the related work into two main directions: (1) Traditional approaches, including handcrafted-feature methods, supervised deep learning, and open-set detection techniques; and (2) self-supervised approaches that rely on pretraining followed by task-specific fine-tuning.

2.1. Traditional approaches

Early network intrusion detection methods typically rely on expert-crafted handcrafted features (e.g., flow statistics, packet size distributions) and use traditional classifiers for training. Tharewal et al. [24] employed LightGBM-based feature selection with a three-layer hidden structure, while AppScanner [11] uses packet size statistics to train a Random Forest classifier. FlowPrint [12] leverages certificate and temporal features, building traffic fingerprints via clustering. These methods require manual design, which is highly dependent on the knowledge of domain experts. To improve generalization, some studies have turned to deep learning to automatically extract features from raw traffic bytes. DeepPacket [25] uses a convolutional neural network to capture spatial features from raw payloads, while Fs-Net [26] employs a recurrent neural network to automatically learn contextual features from packet length sequences in traffic. On the other hand, almost all widely cited traffic classification models are trained and evaluated under a closed-set assumption, which inherently prevents them from handling traffic belonging to classes unseen during training. Some approaches attempt to identify unknown classes by incorporating EVT or confidence calibration mechanisms. Zhang et al. [27] construct the Open-CNN model by fitting the activation values of known attack classes to a Weibull distribution, achieving unknown attack detection.

However, in IoT environments, labeled attack samples are extremely scarce, which limits the effectiveness of the aforementioned supervised learning approaches.

2.2. Self-supervised approaches

Several recent approaches adopt self-supervised pretraining to learn network traffic representations, alleviating the need for labeled data. PERT [14], and ET-BERT [15] convert raw traffic bytes into text tokens and apply masked language modeling (MLM) to learn representations from payloads, followed by task-specific fine-tuning. Similarly, Netmamba [16] employs a masked autoencoder (MAE) [28] to encode traffic and capture dependencies by reconstructing masked patches. However, these methods directly feed byte-level traffic into the sequential models, ignoring packet-level features, which may lead to overfitting on specific data distributions. To address these limitations, YaTC [38] employs dual encoders at both packet and flow granularities during fine-tuning. However, both encoders rely on the same Transformer architecture, limiting the model’s capacity to capture granularity-specific traffic characteristics. Moreover, the $\mathcal{O}(n^2)$ computational cost of self-attention [29] makes real-time intrusion detection in IoT networks challenging.

In summary, although self-supervised pretraining helps reduce the need for labeled data, current methods still face key challenges: they often use the same model structure for different traffic levels, miss important packet-level details, and require too much computation due to heavy sequence modeling. As a result, they are not well suited for real-time intrusion detection in resource-limited IoT networks.

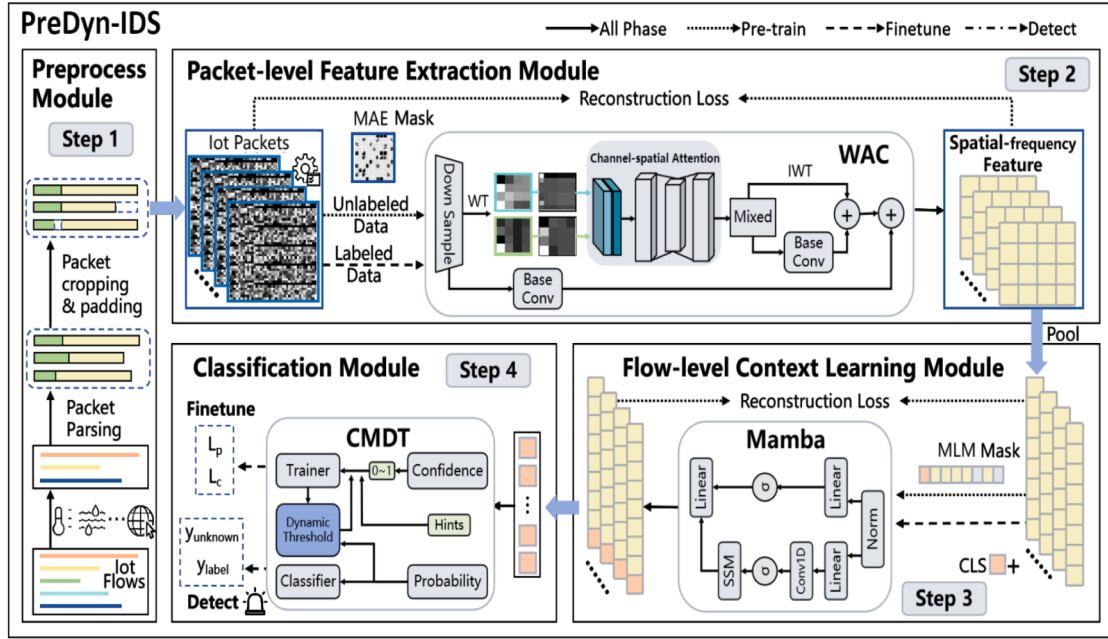


Fig. 1. Overview of PreDyn-IDS Framework. The overall procedure can be summarized as follows: ①The preprocess module processes the original traffic by converting packets into a 2D matrix. ② The packet-level feature extraction module uses Wavelet Attention Convolution (WAC) to extract features with spatial-frequency characteristics. ③The flow-level context learning module feeds the same-flow packet features into a Mamba to capture contextual dependencies and generate flow-level feature vectors. ④ The confidence-based classifier separates unknown categories of traffic and classifies known types by the Confidence Mechanism of Dynamic Threshold (CMTD), achieving accurate and reliable traffic detection.

3. The PreDyn-IDS framework

3.1. Model overview

Our PreDyn-IDS is designed to directly address the three key challenges in open-world IoT intrusion detection: high model complexity, label scarcity, and the need for robust unknown traffic identification. The framework comprises four core components—data processing, packet-level feature extraction, flow-level context learning, and confidence-enhanced classification—as illustrated in Fig. 1. We detail each component in the following subsections.

3.2. Data preprocessing

To extract stable and discriminative features from raw network traffic, we design a standardized data preprocessing pipeline that converts raw PCAP files into 2D matrix representations suitable for deep learning models. We first filter out non-IP traffic (e.g., ARP, DHCP) and split the remaining traffic into individual flows using the 5-tuple—(source IP, source port, destination IP, destination port, protocol)—via SplitCap [30]. To distinguish independent sessions that share the same 5-tuple, we apply a packet inter-arrival time threshold T : if the interval between two consecutive packets exceeds T , a new session is initiated. Flows containing fewer than M packets are discarded to ensure data quality. For each packet within a retained flow, we strip the Ethernet header and anonymize IP addresses and port numbers to prevent the model from overfitting to network-specific metadata [31]. The packet header and payload are then truncated or zero-padded to fixed lengths of N_h and N_p , respectively, and each processed packet is reshaped into a 2D matrix that serves as input to subsequent modules.

3.3. Packet-level feature extraction

IoT traffic packet sequences exhibit inherent structural and statistical patterns. To capture these patterns, we designed the Wavelet Attention

Convolution (WAC) module to extract fine-grained, packet-level features with joint spatial-frequency characteristics, as shown in step 2 of Fig. 1. Unlike traditional convolution, which can only extract single-scale spatial features, the WAC is designed with multi-frequency decomposition and feature enhancement in mind. It integrates the Haar Wavelet Transform [32] and Channel-Spatial Attention [33] through a sequential process. The design principle of WAC is detailed below.

First, we use the Haar Wavelet Transform to decompose the input packet matrix into four frequency bands. The low-frequency component (X_{LL}) captures the overall, smooth structural features of the traffic, while the high-frequency components capture abrupt changes and details. Considering the one-dimensional nature of network traffic data, where the most significant patterns typically manifest horizontally, we place special emphasis on the horizontal high-frequency component (X_{LH}). Then, the concatenated feature map X_{fusion} is fed into the Channel-Spatial Attention (CSA) mechanism. The CSA module weights the fused features, learning to enhance the more informative ones while suppressing irrelevant or noisy features, thereby strengthening the feature representation. The enhanced feature map X_{atten} from the CSA module, in turn, serves as the input for the next level of wavelet transform, allowing for deeper, multi-scale feature extraction. Finally, the enhanced features from the different frequency bands are reconstructed via the Inverse Wavelet Transform (IWT) and combined with the output of a base convolution through a residual connection to generate the final, information-rich joint spatial-frequency feature map $Z(0)$. Algorithm 1 outlines the operational process of the WAC block forward pass.

Given a packet matrix $X \in \mathbb{R}^{C \times H \times W}$, the WAC first initializes the input tensor X as $X_{LL}^{(0)}, X_{LH}^{(0)}$ and performs a standard convolution on $X_{LL}^{(0)}$. In each subsequent stage, $X_{LL}^{(i)}$ and $X_{LH}^{(i)}$ are concatenated and processed by the CSA mechanism to enhance feature expressiveness. The $X_{atten}^{(i)}$ is then recursively decomposed via multiple Wavelet Transform into four subbands: $X_{LL}^{(i)}, X_{LH}^{(i)}, X_{HL}^{(i)}, X_{HH}^{(i)}$, then perform depth-wise convolution on different frequency mappings to obtain $Y_{LL}^{(i)}, Y_{LH}^{(i)}, Y_{HL}^{(i)}, Y_{HH}^{(i)}$. We adopt the Haar wavelet basis due to its opti-

Algorithm 1 WAC Block: hierarchical decomposition and reconstruction.

Input: Input feature map $X \in \mathbb{R}^{C \times H \times W}$

Output: Refined spatio-frequency feature $Z \in \mathbb{R}^{C \times H \times W}$

```

// Stage I: Initial Feature Extraction
1:  $X_{LL}^{(0)}, X_{LH}^{(0)} \leftarrow X$ 
2:  $Y_{LL}^{(0)} \leftarrow \text{Conv}(W^{(0)}, X_{LL}^{(0)})$ 

// Stage II: Multi-level Wavelet Decomposition (Forward Pass)
3: for  $i = 1$  to  $\ell$  do
4:    $X_{fusion} \leftarrow \text{Concat}(X_{LL}^{(i-1)}, X_{LH}^{(i-1)})$   $\triangleright$  Use features from previous level
5:    $X_{atten} \leftarrow \text{CSA}(X_{fusion})$   $\triangleright$  Channel-Spatial Attention
6:    $\{X_{LL}^{(i)}, X_{LH}^{(i)}, X_{HL}^{(i)}, X_{HH}^{(i)}\} \leftarrow \text{WT}(X_{atten})$ 
7:    $\{Y_{LL}^{(i)}, Y_{LH}^{(i)}, Y_{HL}^{(i)}, Y_{HH}^{(i)}\} \leftarrow \text{Conv}(W^{(i)}, \{X_{LL}^{(i)}, \dots, X_{HH}^{(i)}\})$ 
8: end for

// Stage III: Progressive Inverse Reconstruction
9:  $Z^{(\ell+1)} \leftarrow \mathbf{0}$ 
10: for  $i = \ell$  down to 1 do
11:    $Z^{(i)} \leftarrow \text{IWT}(Y_{LL}^{(i)} + Z^{(i+1)}, Y_{LH}^{(i)}, Y_{HL}^{(i)}, Y_{HH}^{(i)})$ 
12: end for

// Stage IV: Final Residual Fusion
13:  $Z^{(0)} \leftarrow Y_{LL}^{(0)} + Z^{(1)}$ 
14: return  $Z^{(0)}$ 

```

mal trade-off between performance and computational efficiency. This process achieves simultaneous frequency-selective filtering and spatial downsampling across both low- and high-frequency domains. To combine the outputs of different stages, we recursively perform the wavelet inverse transform, $Z^{(i)}$ represents the aggregated output from the i -th layer and below. At last, we add $Y_{LL}^{(0)}$ with $Z^{(1)}$ by residual concatenation to get the final spatial-frequency feature $Z^{(0)}$ of the packet matrix. $Y_{LL}^{(0)}$ is the information without performing spatial-frequency transformations. Through the WAC module, PreDyn-IDS can extract more robust and discriminative per-packet features than conventional CNNs, laying a solid foundation for the subsequent flow-level analysis.

3.4. Flow-level context learning

To capture macroscopic communication behaviors across packets within a flow, we design a lightweight and efficient flow-level context learning module based on the Mamba state-space model. By operating directly on the packet-level features extracted by the WAC module—rather than raw bytes or long token sequences—our approach significantly reduces sequence length and eliminates the need for costly embedding layers and vocabulary construction.

As illustrated in step 3 of Fig. 1, we utilize the feature vectors from the first 8 packets of a flow to form the input sequence. This sequence length is substantially shorter than the 256 or 512 tokens typically required in conventional approaches, thereby facilitating real-time processing of IoT traffic. To preserve the sequential order of packets, we incorporate learnable positional embeddings. The final input representation X_0 is formulated as follows:

$$X_0 = [E_{cls}; E_{p1}; E_{p2}; \dots; E_{p8}] + E_{pos} \quad (1)$$

where $E_{cls} \in \mathbb{R}^{1 \times E}$ is a classification token used to aggregate flow-wide information, $E_{pi} \in \mathbb{R}^{1 \times E}$ denotes the features extracted from the i -th packet, and E_{pos} represents the learnable positional embeddings.

The core of this module consists of a series of Mamba blocks. For a given package-level feature sequence $X_{t-1} \in \mathbb{R}^{B \times L \times E}$ from the $(t-1)$ -th block, the forward pass begins with normalization followed by a dual-

branch linear projection to obtain the initial hidden states x and z :

$$\begin{aligned} x &= \text{SiLU}(\text{Conv1d}(\text{Linear}^x(\text{RMSNorm}(X_{t-1})))) \\ z &= \text{Linear}^z(\text{RMSNorm}(X_{t-1})) \end{aligned} \quad (2)$$

where RMSNorm denotes Root Mean Square Normalization, and the x branch incorporates a causal 1-D convolution and SiLU activation to capture local temporal dependencies. To implement the selective mechanism, the input-dependent parameters Δ (step size), B , and C are dynamically generated from x :

$$\begin{aligned} \Delta &= \text{Softplus}(\text{Parameter}^\Delta + \text{Linear}^\Delta(x)), \\ B, C &= \text{Linear}^{B,C}(x) \end{aligned} \quad (3)$$

These parameters are used to discretize the continuous state-space matrices $\{A, B\}$ into their discrete counterparts $\{\bar{A}, \bar{B}\}$ via the zero-order hold (ZOH) rule. Subsequently, the hardware-aware selective scanning mechanism (SSM) is applied to produce the response y :

$$y = \text{SSM}(\bar{A}, \bar{B}, C)(x) \quad (4)$$

Finally, the output of the t -th block, X_t , is formed by gating y with the signal z and applying a residual connection to the original input:

$$X_t = \text{Linear}^o(y \odot \text{SiLU}(z)) + X_{t-1} \quad (5)$$

where $\odot$ denotes the element-wise product. This architecture ensures linear time complexity relative to the sequence length, satisfying the low-latency requirements of IoT traffic classification.

3.5. Confidence mechanism of dynamic threshold

Most traffic classifiers rely solely on softmax probabilities and cannot recognize unknown classes, often misclassifying traffic from new device connections or novel attacks as known categories in IoT networks. To address this limitation, we propose the Confidence Mechanism with Dynamic Threshold (CMDT), which incorporates confidence estimation during both fine-tuning and detection to effectively mitigate forced misclassification in open IoT environments, as shown in step 4 of Fig. 1. In what follows, we first describe how confidence is integrated into the fine-tuning and detection workflow, and then present the underlying principle of dynamic confidence thresholding

(1) Confidence Integration in Fine-Tuning and Detection Workflow:

During fine-tuning, the input is E_{cls} —a holistic traffic flow embedding from the flow-level context learning module. The classification head is designed with two parallel branches. One branch outputs the conventional class probability distribution, by flattening the E_{cls} embedding and feeding it into a Multilayer Perceptron (MLP) to obtain the predicted distribution p_i , as shown in Eq. 6. The other branch uses a Sigmoid function to output a confidence score, c , ranging between 0 and 1, as shown in Eq. 7. A higher value of c indicates that the model is more "familiar" with the given sample. When the model encounters a flow from an unlearned category, it will output a low confidence score, thereby preventing the flow from being classified as any of the known labels. On the other hand, to train the confidence score, the model introduces a "hint" mechanism. During training, we use the confidence score, c , to dynamically fuse the model's prediction, p , with the true label, y . As shown in Eq. 8, p_i denotes the predicted probability of the flow, and y_i denotes the ground-truth label. When the model is uncertain, the confidence score c will be lower, and the model will leverage the true label y to aid its decision. Over successive training epochs, the confidence for the classes of interest gradually increases.

$$p_i = \text{Softmax}(\text{MLP}(E_{cls})) \quad (6)$$

$$c = \text{Sigmoid}(\text{MLP}(E_{cls})) \quad (7)$$

$$p'_i = c \cdot p_i + (1 - c) \cdot y_i \quad (8)$$

Table 1

Hyperparameters of the dynamic threshold adjustment algorithm.

Symbol	Description
α_{short}	Smoothing factor for short-term EMA
α_{mid}	Smoothing factor for mid-term EMA
α_{long}	Smoothing factor for long-term EMA
growth_rate	the rate of increase of the target threshold
decay_rate	the rate of decrease of the target threshold
θ_{max}	Upper bound of the target threshold
θ_{min}	Lower bound of the target threshold
ϵ	Threshold deviation condition for triggering updates during system stability
step	Controls the magnitude of changes in the threshold during stable periods
μ_{update}	Momentum factor used to dynamically adjust the threshold update process

where E_{cls} denotes the representation of the input flow, p_i denotes the predicted probability of the flow belonging to the i_{th} class, and y_i represents the ground-truth probability. The refined probability p'_i is obtained through ground-truth prompting, where c indicates the model confidence score of the flow. MLP(.) denotes a multilayer perceptron.

During the detection phase, if an input flow's confidence score, c , is below the finally determined dynamic threshold, θ_{dynamic} , the flow is marked as Unknown. Otherwise, the model uses the class with the highest probability from the Softmax output as its final prediction. (see Eq. 9). Through the CMDT module, PreDyn-IDS not only accurately classifies known traffic but also reliably identifies and isolates unknown traffic, significantly enhancing its value for deployment in real-world network environments.

$$r = \begin{cases} \text{unknown}, & \text{if } c < \delta_c \\ \arg \max_{i \in n_c} p'_i, & \text{otherwise} \end{cases} \quad (9)$$

(2) Principle of Dynamic Confidence Thresholding

Given that a fixed confidence threshold is difficult to adapt to different datasets and training stage, we propose a CMDT that tracks the short, medium, and long-term trends of the confidence score during training using a multi-time-scale Exponential Moving Average (EMA) and adaptively adjusts the threshold, θ_{dynamic} , as detailed in Algorithm 2. The hyperparameters in Algorithm 2 are shown in Table 1.

Specifically, first, based on obtaining the mean confidence value C_{mean} and the proportion of low-confidence samples LR , calculate the short-term, medium-term, and long-term trends, respectively. Namely, the short-term EMA ($C_{\text{short}}^{(t)}, LR_{\text{short}}^{(t)}$), the medium-term EMA ($C_{\text{mid}}^{(t)}, LR_{\text{mid}}^{(t)}$), and the long-term EMA ($C_{\text{long}}^{(t)}, LR_{\text{long}}^{(t)}$), which can effectively capture the overall trend of confidence changes. Next, the trend indicators ($C_{\text{trend}}, LR_{\text{trend}}$) are calculated by comparing the difference between the short-term and long-term EMA. The confidence trend is used to determine whether the model's prediction confidence is increasing or decreasing, based on which the target threshold θ_{target} is adaptively adjusted. When the confidence is rising ($C_{\text{trend}} > \delta$), the threshold θ_{target} is increased to impose a stricter filtering criterion, encouraging the model to focus on more challenging samples. Conversely, when the confidence is declining ($C_{\text{trend}} < -\delta$), the threshold θ_{target} is lowered to relax the filtering condition, preventing excessive sample exclusion. When the prediction confidence trend stabilizes, the algorithm calculates the deviation between the current mid-term low-confidence ratio $LR_{\text{mid}}^{(t)}$ and the trend reference value LR_{trend} . If this deviation exceeds the deviation tolerance ϵ , the θ_{target} is slightly adjusted according to the deviation direction and a predefined step size, to improve responsiveness to local fluctuations. Otherwise, a weighted average of the mid-term and long-term confidence scores is used as the target threshold θ_{target} , ensuring smooth updates and training stability. Then both the (C_{trend} and the LR_{trend}) are used to compute a momentum value μ , which in turn

guides the update of the dynamic threshold θ_{dynamic} by combining the target threshold θ_{target} with the current one θ_t , preventing sudden or drastic changes in the threshold.

Algorithm 2 Dynamic threshold adjustment algorithm.

Input: Confidence_set C_t , current_threshold θ_t

Output: Updated threshold θ_{dynamic}

```

1: Set hyperparameters:  $\alpha_{\text{short}} = 0.7, \alpha_{\text{mid}} = 0.9, \alpha_{\text{long}} = 0.95,$ 
   growth_rate = 1.05,  $\theta_{\text{max}} = 0.995,$  decay_rate = 0.95,  $\theta_{\text{min}} = 0.85,$ 
    $\epsilon = 0.1, \text{step} = .05, \mu_{\text{update}} = 0.3$ 
2:  $C_{\text{mean}} \leftarrow \text{average}(C_t)$   $\triangleright$  Compute current mean confidence:
3:  $LR = \frac{|\{c \in C_t : c < \theta_t\}|}{|C_t|}$   $\triangleright$  Compute low-confidence ratio
4:  $C_{\text{short}}^{(t)} = \alpha_{\text{short}} \cdot C_{\text{mean}} + (1 - \alpha_{\text{short}}) \cdot C_{\text{short}}^{(t-1)}$   $\triangleright$  Compute EMA
5:  $C_{\text{mid}}^{(t)} = \alpha_{\text{mid}} \cdot C_{\text{short}}^{(t)} + (1 - \alpha_{\text{mid}}) \cdot C_{\text{mid}}^{(t-1)}$ 
6:  $C_{\text{long}}^{(t)} = \alpha_{\text{long}} \cdot C_{\text{mid}}^{(t)} + (1 - \alpha_{\text{long}}) \cdot C_{\text{long}}^{(t-1)}$ 
7:  $LR_{\tau}^{(t)} = \alpha_{\tau} \cdot LR_{\tau}^{(t-1)} + (1 - \alpha_{\tau}) \cdot LR, \forall \tau \in \{\text{short, mid, long}\}$ 
8:  $x_{\text{trend}} = x_{\text{short}}^{(t)} - x_{\text{long}}^{(t)}, \forall x \in \{C, LR\}$   $\triangleright$  Compute trend
9: if  $C_{\text{trend}} > \delta$  then  $\triangleright \delta$  is the trend threshold
10:    $\theta_{\text{target}} = \min(C_{\text{mid}}^{(t)} \cdot \text{growth\_rate}, \theta_{\text{max}})$   $\triangleright$  Increase threshold
11: else if  $C_{\text{trend}} < -\delta$  then
12:    $\theta_{\text{target}} = \max(C_{\text{mid}}^{(t)} \cdot \text{decay\_rate}, \theta_{\text{min}})$   $\triangleright$  Decrease threshold
13: else
14:   ratio_deviation =  $LR_{\text{mid}}^{(t)} - LR_{\text{trend}}$ 
15:   if ratio_deviation  $> \epsilon$  then  $\triangleright \epsilon$  is the deviation tolerance
16:      $\theta_{\text{target}} = \theta_t \cdot (1 + \text{sign}(\text{ratio\_deviation}) \cdot \text{step})$ 
17:   else
18:      $\theta_{\text{target}} = \alpha_{\text{mid}} \cdot C_{\text{mid}}^{(t)} + \alpha_{\text{long}} \cdot C_{\text{long}}^{(t)}$ 
19:   end if
20: end if
21:  $\mu = \mu_{\text{update}} \cdot C_{\text{trend}} + (1 - \mu_{\text{update}}) \cdot LR_{\text{trend}}$   $\triangleright \mu$  is momentum
22:  $\theta_{\text{dynamic}} = \mu \cdot \theta_t + (1 - \mu) \cdot \theta_{\text{target}}$ 
23: return  $\theta_{\text{dynamic}}$ 

```

4. Experiment setup

4.1. Datasets and metrics

To further validate the generality of our model, experiments are also conducted on the following datasets: Edge-IIoT [4]: a large-scale, real-world IoT dataset featuring diverse attack scenarios and normal traffic from operational technology and IT environments; ISCXVPN [34]: a traffic classification benchmark containing encrypted application-layer traffic over VPN tunnels, widely used to evaluate deep packet inspection under encryption; ISCXTor [35]: This dataset includes traffic from 16 applications anonymized through the Tor network.

We employ four standard classification task evaluation metrics to measure model performance: Accuracy(AC), Precision(PR), Recall(RC), and F1-Score.

4.2. Implementation details and training strategy

Our model was implemented using the PyTorch 2.1.2 framework and trained on a server equipped with NVIDIA A800 GPUs. The training strategy follows the pretraining and fine-tuning paradigm and the hyperparameters are in Table 2.

Pretraining Phase: We consolidated a total of 203,817 unlabeled traffic samples from four datasets (ISXBot2014, VNAT, ISCXVPN, and ISCXTor) to create the pretraining dataset. During this phase, the model was trained for 90,000 steps with a batch size of $B = 128$, a learning rate of $1e-3$, and the AdamW optimizer combined with a cosine annealing learning rate scheduler.

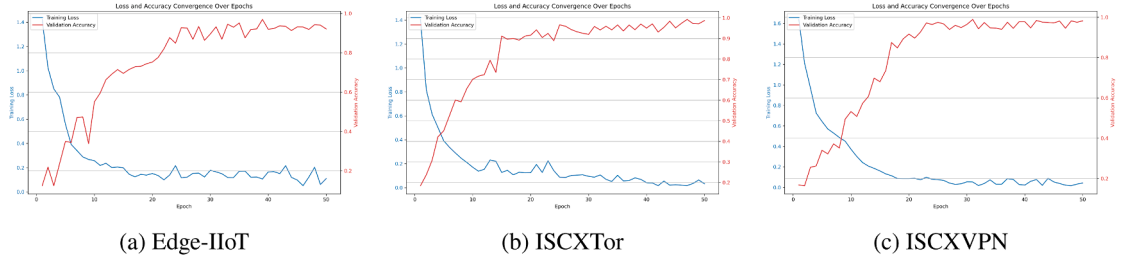


Fig. 2. Training convergence curves.

Table 2

Model configuration parameters and their values.

Variable	Value	Variable	Value	Variable	Value
M	8	B_{pre}	128	B_{fine}	64
N_h	48	$Step_{pre}$	90,000	E_{poch}_{fine}	50
N_p	976	lr_{pre}	1e-3	lr_{fine}	2e-4
t	0.256	r_p	0.6	r_f	0.8

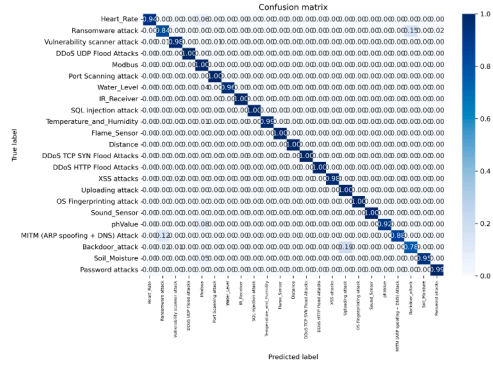


Fig. 3. Confusion matrix on the edge-IIoT dataset.

Fine-tuning Phase: Building on the pretrained model parameters, supervised learning is conducted on the labeled data for each downstream task. We partitioned each dataset into training, validation, and test sets using an 8:1:1 ratio. The batch size is reduced to $B=64$, the learning rate is adjusted to $2e-4$, and the model is trained for a total of 50 epochs.

4.3. Performance comparison with SOTA methods

We conduct a comprehensive comparison between our model and existing methods on four public benchmark datasets, evaluating performance across two fundamental tasks in network traffic analysis: anomaly detection and multi-class classification. The training convergence curves of PreDyn-IDS are shown in Fig. 2.

(1) Anomaly Detection Results:

Fig. 3 shows that our model demonstrates outstanding overall performance on the IoT network traffic classification task: it achieves a test accuracy of 97.08%, along with a precision of 94.66%, recall of 97.07%, and F1-score of 94.19%. The confusion matrix reveals that the vast majority of traffic classes—including normal services such as Modbus and Temperature_and_Humidity, as well as various attacks like DDoS, Ransomware, and SQL injection—are classified with high accuracy, as evidenced by diagonal entries close to 1.0. Only minimal cross-class confusion occurs (e.g., MITM attacks occasionally misclassified as Backdoor attacks), indicating that the model maintains strong discriminative capability even among semantically or behaviorally similar traffic types. The high recall further implies an extremely low false-negative (miss) rate.

(2) Multi-class Classification Results:

On ISCVPN-APP, ISCVPN-Service, and ISCXTor, PreDyn-IDS achieves F1 scores of 91.32%, 97.42%, and 97.01%, respectively, as shown in Table 3. This performance can be attributed to the following aspects: Compared with Transformer-based methods (e.g., ET-BERT, PERT, TrafficFormer), which offer strong representational capacity but suffer from high computational cost, PreDyn-IDS adopts a lightweight WAC module and a linear-complexity Mamba architecture. It achieves the highest precision on both Edge-IIoT and ISCXTor, and the second-highest on ISCVPN, while significantly reducing model size compared to Transformer-based methods. Compared with label-dependent supervised models (e.g., DeepPacket, FS-Net) or handcrafted-feature methods (e.g., AppScanner, FlowPrint), which have limited generalization, PreDyn-IDS employs packet-level MIM and flow-level MLM self-supervised pretraining to learn general representations from unlabeled traffic, alleviating the reliance on labeled data. Compared with existing methods that assume a closed set of classes (e.g., YATC, Net-Mamba), which cannot handle unknown traffic, PreDyn-IDS introduces the CMDT mechanism to accurately classify known categories and reliably isolate unknown samples, thereby improving classification accuracy.

4.4. Ablation study

To validate the effectiveness of each component in PreDyn-IDS, we conduct ablation studies on the ISCVPN-Service and ISCXTor datasets. The experimental results are shown in Table 4. Removing the entire pretraining phase causes a significant drop in performance: the F1-score decreases from 97.42% to 86.44% on ISCVPN-Service and from 97.01% to 91.77% on ISCXTor, demonstrating the importance of learning general-purpose traffic representations from unlabeled data. Ablating either the packet-level MIM or flow-level MLM task also leads to notable degradation, indicating their complementary roles in capturing multi-scale features. Replacing the WAC module with a standard CNN block (w/o WTconv) results in the most severe performance decline, reducing the F1-score to 85.15% on ISCVPN-Service and 90.41% on ISCXTor, which confirms that joint spatial-frequency feature extraction is essential for traffic classification. Removing the Mamba module reduces the F1-score by 3.64% and 2.64% on the two datasets, respectively, highlighting the value of modeling contextual dependencies within flow. Furthermore, we investigate the necessity of absolute position embeddings within the Mamba framework (w/o E_{pos}). The results show that removing E_{pos} leads to a drop in F1-score to 95.82% and 96.14%, respectively. This proves that explicit positional encoding provides a vital global coordinate system that complements Mamba's recurrent mechanism, enabling the model to better distinguish the structural order of packets in encrypted sessions. Finally, disabling CMDT leads to a noticeable drop in both accuracy and F1-score—down to 88.92% and 87.17% on ISCVPN-Service, and 87.61% and 87.26% on ISCXTor—indicating that CMDT not only plays a central role in handling unknown traffic but also assists in training by optimizing the decision boundaries for known classes.

Table 3
Comparison results on ISCXVPN-APP, ISCXVPN-Service, and ISCXTor.

Dataset	ISCXVPN-APP(Label = 17)				ISCXVPN-Service(Label = 12)				ISCXTor(Label = 16)			
Method	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
Appscanner [11]	0.7724	0.7146	0.6703	0.6918	0.7182	0.7339	0.7059	0.7197	0.6722	0.3756	0.4257	0.3913
FlowPrint [12]	0.8767	0.6697	0.6443	0.6531	0.7962	0.8042	0.7641	0.7820	0.9092	0.3820	0.3654	0.3654
FS-Net [26]	0.6647	0.4819	0.4685	0.4737	0.7205	0.7502	0.6952	0.7131	0.6071	0.5080	0.4536	0.4590
Deeppacket [25]	0.8892	0.8784	0.8679	0.8723	0.8946	0.8858	0.8819	0.8779	0.7449	0.7549	0.7499	0.7473
Pert [14]	0.8229	0.7092	0.6901	0.6992	0.9352	0.9400	0.9332	0.9368	0.7682	0.4424	0.4369	0.4345
Trafficformer [18]	0.8000	0.7675	0.7036	0.6959	0.9247	0.9268	0.9167	0.9205	0.9235	0.9077	0.9409	0.9240
YATC [17]	0.7310	0.6004	0.6069	0.6034	0.8356	0.8163	0.8186	0.8122	0.9314	0.9167	0.9288	0.9227
NetMamba [16]	0.8735	0.8539	0.8663	0.8597	0.9441	0.9360	0.9143	0.9201	0.9565	0.9718	0.9562	0.9639
ET-Bert(flow) [15]	0.8519	0.7508	0.7158	0.7306	0.9729	0.9756	0.9709	0.9733	0.8311	0.5564	0.6316	0.5886
Ours	0.8989	0.9231	0.9025	0.9132	0.9867	0.9615	0.9877	0.9742	0.9806	0.9700	0.9700	0.9701

Table 4
Ablation analysis of PreDyn-IDS components.

Method	ISCXVPN-Service		ISCXTor	
	AC	F1	AC	F1
PreDyn-IDS (full model)	0.9867	0.9742	0.9806	0.9701
w/o MIM	0.9087	0.9108	0.9369	0.9337
w/o MLM	0.9354	0.9383	0.9536	0.9513
w/o pretraining	0.8711	0.8644	0.9102	0.9177
w/o WTconv	0.8589	0.8515	0.9074	0.9041
w/o Mamba	0.9360	0.9378	0.9476	0.9437
w/o E_{pos}	0.9654	0.9582	0.9691	0.9614
w/o CMDT	0.8892	0.8717	0.8761	0.8726

Table 5
Comparison of model parameters and computational complexity.

Model	Params (M)	FLOPs (G)
FS-Net [26]	5.3	4.9
ET-BERT [15]	187.4	136.4
YaTC [17]	2.3	2.1
NetMamba [16]	2.2	2.0
PreDyn-IDS	2.1	1.8

4.5. Inference efficiency evaluation

To verify the operational efficiency of PreDyn-IDS in processing network traffic, this section provides a quantitative analysis of the computational overhead and real-time processing capabilities.

4.5.1. Model complexity analysis

As illustrated in Table 5, PreDyn-IDS demonstrates advantages in both parameter count (Params) and computational complexity (FLOPs). The total parameters of PreDyn-IDS are only 2.1 M; in contrast, FS-Net possesses 5.3 M parameters, while the heavy-duty ET-BERT reaches as high as 187.4 M parameters—approximately 89 times that of our framework. This makes PreDyn-IDS more competitive in terms of memory footprint and storage space. Regarding computational complexity, since PreDyn-IDS adopts the latest Mamba-2 (SSM-V2) architecture, which achieves a deep integration of SSM and matrix multiplication by introducing Structured State Space Duality (SSD), it outperforms the first-generation Mamba-based NetMamba (2.0 G FLOPs) in both parallel processing efficiency and theoretical overhead, with its FLOPs further reduced to 1.8 G. Such lower resource requirements help reduce system CPU load, fully demonstrating its practical value in network anomaly detection tasks.

4.5.2. Trade-off between inference speed and accuracy

To further evaluate the comprehensive performance of the model in real-time traffic monitoring, Fig. 4 illustrates the relationship between inference speed (samples per second) and detection accuracy for PreDyn-IDS and baseline models. Experimental results show that

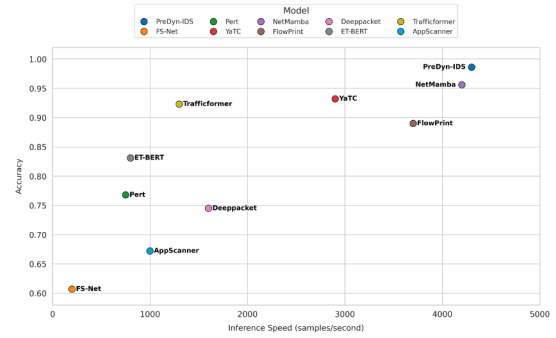


Fig. 4. The inference efficiency comparison.

PreDyn-IDS significantly enhances system throughput while maintaining superior detection precision. Specifically, the inference speed of PreDyn-IDS reaches approximately 4300 samples/second, ranking highest among all evaluated models. In contrast, the RNN-based FS-Net exhibits the lowest throughput due to serial processing constraints. Transformer-based models, such as ET-BERT and Trafficformer, show inference speeds clustered between 1000 and 1500 samples/second, primarily limited by the complexity of self-attention. Notably, while NetMamba also displays impressive efficiency, PreDyn-IDS leverages Mamba-2's optimized matrix operations to achieve even higher throughput. This combination of high throughput and precision proves that PreDyn-IDS can effectively handle real-time traffic analysis in large-scale environments, achieving an optimal balance between resource consumption and detection performance.

4.6. Robustness evaluation in an open-world scenario

To simulate the "open-world" scenario, where the presence of unknown traffic is inevitable in real-world networks, we conducted an experiment on the ISCXTor dataset. We fine-tuned the model on six traffic classes, treating the remaining ten types as unknown traffic (the ISCXTor dataset contains 16 types in total). We designed three experimental settings: (a) Fully Known Classification, where the model is tested only on accurately labeled, known-class traffic to establish a theoretical performance upper bound; (b) Traditional Closed-Set Classification, where all unknown samples are forcibly assigned to one of the known classes; and (c) Open-World Classification with Our Proposed Method, where unknown traffic is filtered out by the confidence mechanism and labeled as OOD_class.

The results are illustrated in Fig. 5. Under the traditional closed-set setting (Fig. 5b), the model is forced to misclassify unknown traffic into one of the six known categories. This results in a chaotic confusion matrix and a severe degradation in performance, starkly exposing the vulnerability of conventional methods in practical deployment scenarios. In contrast, under our open-world setting (Fig. 5c), enabled by the CMDT, our model successfully identifies and isolates the vast majority



Fig. 5. Confusion matrix comparison under varying classification settings.

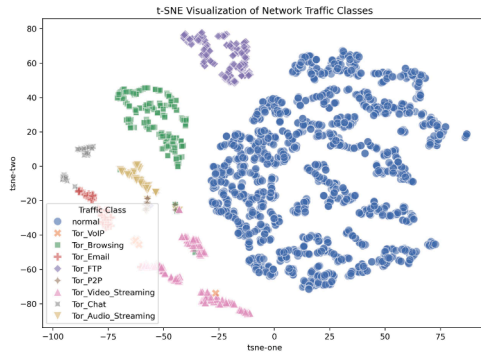


Fig. 6. The visualization of packet-level features on ISCXTor.

of unknown traffic (labeled as OOD_class). Meanwhile, the classification accuracy for the known classes remains almost unaffected, with performance closely approaching that of the ideal, fully supervised scenario (Fig. 5a). This comparative experiment intuitively demonstrates the robustness and practical value of our framework in real-world, dynamic network environments, addressing a core pain point of traditional models in practical deployment.

4.7. Feature visualization and unknown traffic clustering

To evaluate the performance of the packet-level feature extraction module based on WAC, particularly its ability to generate highly distinctive embeddings for distinguishing different types of network traffic, we utilize t-SNE for dimensionality reduction and visualization. As can be seen in Fig. 6, after processing with WAC, the packet-level features of various traffic categories are clustered with distinct boundaries, especially in the ISCXTor dataset. The experimental results indicate that our packet-level feature extraction module can successfully capture and characterize the essential features of different traffic categories. It provides a solid representation foundation for subsequent flow-level contextual modeling.

To validate the effectiveness of clustering unknown traffic, we applied an unsupervised clustering workflow to the unknown subsets isolated by the CMDT mechanism across three datasets: ISCXTor, ISCXVPN-App, and ISCXVPN-Service. The workflow first employs PCA for dimensionality reduction, followed by density-based clustering using HDBSCAN with an adaptive minimum cluster size determined through a semi-automatic search, which eliminates the need to pre-specify the number of clusters. Across multiple experimental trials on all datasets, the resulting number of clusters consistently remained within ± 1 of the true class count. As illustrated in Table 6, specific representative categories were designated as "Unknown Categories" for open-set testing (e.g., CHAT and VOIP in ISCXTor, Netflix and P2P in other datasets). In our experimental setup, these categories were entirely excluded during the model training phase to simulate real-world scenarios involving

Table 6

Clustering evaluation results on datasets with specific unknown traffic categories.

Dataset (Unknown Categories)	ARI ($\uparrow$)	NMI ($\uparrow$)	Silhouette ($\uparrow$)
ISCXTor (Unknown: CHAT, VOIP)	0.72	0.75	0.59
ISCVVPN-App (Unknown: Netflix, Skype)	0.79	0.81	0.65
ISCVVPN-Service (Unknown: Email, P2P)	0.76	0.78	0.62

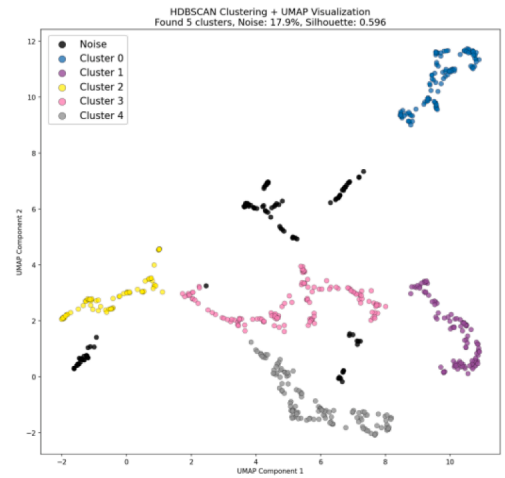


Fig. 7. Unsupervised clustering of CMDT-isolated unknown traffic using HDBSCAN.

zero-day attacks or novel application traffic. Fig. 7 illustrates a representative experimental result (taking ISCXTor as an example) with a silhouette coefficient of approximately 0.59, indicating well-separated and compact clusters. Furthermore, to externally validate the alignment between clustering results and ground truth labels, we incorporated the Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI) as evaluation metrics. PreDyn-IDS achieved ARI values ranging from 0.72 to 0.79 and NMI values between 0.75 and 0.81 across the three datasets. Despite the inherent complexity of encrypted traffic, these robust external validation scores demonstrate that the features extracted by PreDyn-IDS are highly discriminative, yielding cluster structures that closely correspond to true underlying categories across diverse scenarios. This structured grouping of unknown traffic provides a solid foundation for subsequent manual analysis.

4.8. Further analysis

(1) **Frequency-Subband Feature Analysis of the WAC Module** To further understand the advantages of the WAC module in traffic clas-

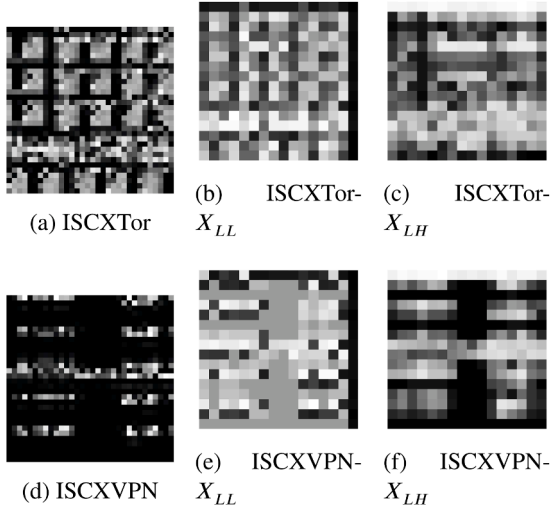


Fig. 8. Feature visualization of ISCXTor and ISCVPN extracted by WAC.

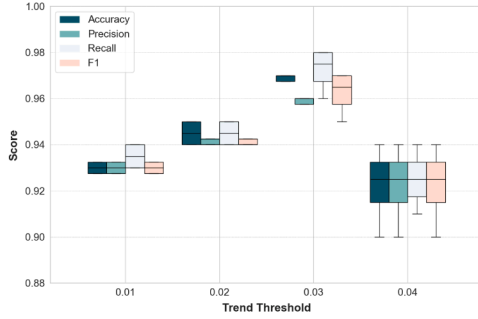


Fig. 9. Sensitivity analysis of trend thresholds.

sification, we visualize the feature representations across different frequency subbands, as shown in Fig. 8. The X_{LL} subband captures the overall structure of the packet through its low-frequency components, reflecting the underlying byte-level patterns of the traffic. The X_{LH} subband focuses on extracting high-frequency variations along the horizontal direction, effectively capturing features such as field boundaries and protocol layer transitions.

(2) Trend Threshold Parameter Experiment

The CMDT requires a suitable trend threshold that controls the frequency of target threshold updates. We experimented with multiple trend threshold settings on the ISCXTor dataset. A lower threshold (e.g., 0.01) makes the mechanism too sensitive, causing the target threshold to oscillate and fail to converge. A higher threshold (e.g., 0.04) makes the mechanism too insensitive, leading to coarse adjustments and potential error accumulation. The results, shown in Fig. 9, indicate that an optimal balance is needed.

(3) Sensitivity Analysis of Packet Number

To evaluate the impact of the input truncation length (i.e., the number of packets extracted per flow) on the performance of PreDyn-IDS and to address concerns regarding long-range dependency modeling and sensitivity to slow-rate attacks, we conducted a sensitivity analysis on the parameter M . The values of M were varied across {4, 6, 8, 10, 12}, and the corresponding F1-Scores were observed on three representative datasets: ISCVPN-Service, ISCVPN-APP, and ISCXTor. As illustrated in Fig. 10, the F1-Scores for all datasets show a significant upward trend as M increases from 4 to 8. This indicates that in the early stages of a flow, increasing the number of packets substantially enriches the contextual information for the Mamba module, enhancing its ability to capture complex traffic patterns, including those associated with slow-rate attacks that rely on long-range dependencies. The model reaches its peak

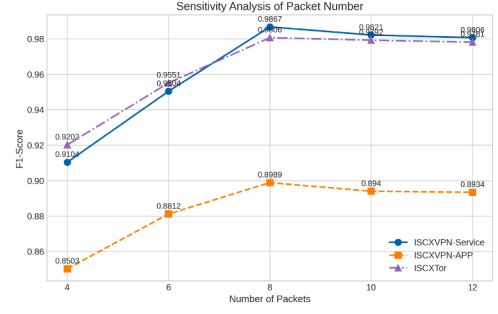


Fig. 10. Sensitivity analysis of packet number.

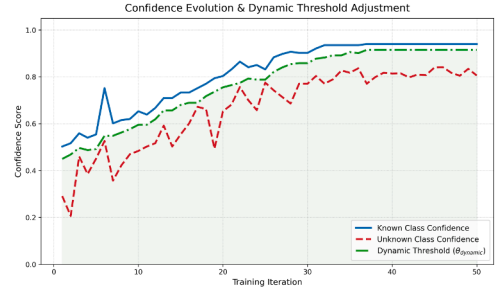


Fig. 11. Confidence evolution and dynamic threshold adjustment.

performance across all datasets when $M = 8$ (e.g., achieving 0.9867 on ISCVPN-Service). However, as M further increases to 10 or 12, the performance plateaus or exhibits slight fluctuations. This phenomenon suggests that for most encrypted traffic classification tasks, the first 8 packets of a flow already contain sufficient fingerprint information for accurate identification. Continuously increasing the truncation length does not yield significant precision gains; instead, it increases the computational overhead of the WAC module and introduces potential feature redundancy. Consequently, PreDyn-IDS adopts $M = 8$ as the default threshold, ensuring robust sensitivity to long-sequence features while maximizing inference efficiency to achieve an optimal balance between performance and cost.

(4) Visualization of Confidence Score Evolution

To validate the effectiveness of our dynamic threshold mechanism in mitigating "blind confidence," we visualized the confidence evolution trajectories for both known and unknown classes. As illustrated in the Fig. 11, the confidence for known classes (blue solid line) exhibits a steady upward trend throughout the training process. In contrast, the confidence for unknown classes (red dashed line) fluctuates significantly during the early stages but gradually converges to a lower level. This indicates that while the model is initially uncertain about unknown samples, the dynamic adjustment allows it to effectively distinguish between known and unknown categories, thereby avoiding the pitfall of misclassifying unknowns as knowns. Furthermore, the dynamic threshold (green dashed line) consistently maintains a boundary between the two confidence scores, mathematically verifying its role in preventing gradient traps.

(5) Sensitivity Analysis of Hyperparameters

To evaluate the sensitivity of PreDyn-IDS to hyperparameters and address concerns regarding hard-coded settings, this section performs a sensitivity analysis on the core parameters of the multi-time-scale EMA update algorithm (Algorithm 2), including the decay rates α_{short} , α_{mid} , α_{long} and the momentum coefficient β . By applying single-variable perturbations within a range of $\pm 20\%$ relative to the initial values, we observed their impact on overall detection performance. As illustrated in Fig. 12, the sensitivity curves for all hyperparameters peak near the 0% perturbation point, validating the optimality of our current configuration. Crucially, the curves for α_{mid} and α_{long} remain relatively

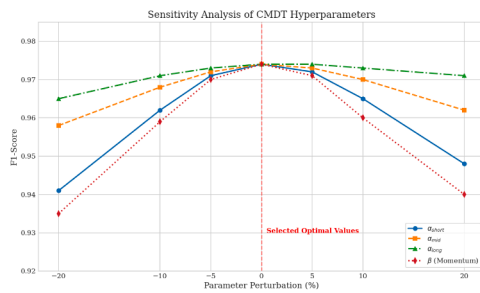


Fig. 12. Sensitivity analysis of CMDT hyperparameters.

markedly flat, with F1-Score fluctuations remaining negligible even under significant parameter variations. This highlights a key advantage of PreDyn-IDS: because the CMDT mechanism employs a dynamic adjustment process, the system can autonomously recalibrate detection thresholds based on real-time traffic characteristics, thereby reducing the dependency on the precision of initial hyperparameter settings. Such adaptive capability effectively mitigates potential biases introduced by manual parameterization and ensures robust performance when managing long-term background traffic. The experimental results demonstrate that PreDyn-IDS maintains stable and high detection accuracy across a wide range of parameter values, confirming the reliability and practical feasibility of the dynamic threshold mechanism in diverse and evolving network environments.

5. Discussion

Our current flow-level context learning module is built upon the first-generation Mamba architecture. In future work, we plan to investigate advanced variants such as Mamba-2, which offers improved hardware-aware optimizations and broader deployment compatibility, thereby extending the applicability of Packet2Flow across diverse IoT platforms. We also aim to explore the potential of integrating a continual learning mechanism that could enable the model to dynamically update its knowledge after deployment, maintaining long-term detection accuracy and relevance without human intervention.

6. Conclusion

We propose a lightweight open-set intrusion detection method for IoT that accurately classifies known attacks and effectively identifies anomalous traffic from unknown threats. First, we design a cross-granularity feature fusion architecture that integrates spatial-frequency features from a lightweight WAC module at the packet level with contextual information from a linear-complexity Mamba model at the flow level, achieving strong representation capability with low computational overhead. Second, to reduce reliance on manual labels, we adopt a pretraining-finetuning strategy based on MAE and MLM to learn generalizable flow representations from abundant unlabeled traffic. Finally, we integrate CMDT during both fine-tuning and inference, using a dynamic confidence threshold to reliably classify known flows and effectively isolate unknown ones. Experiments on IoT and general network traffic datasets show that our method consistently outperforms state-of-the-art baselines in detection accuracy, validating its feasibility and effectiveness.

CRedit authorship contribution statement

Xi Luo: Writing – review & editing, Writing – original draft, Methodology, Conceptualization; **Guohua Yan:** Writing – original draft, Investigation, Data curation; **Lihua Yin:** Supervision, Project administration; **Gaopan Hou:** Validation, Software; **Jingjing Guo:** Resources, Data curation; **Yanqi Song:** Visualization, Formal analysis; **Xiang Li:** Software,

Investigation; **Wenting Wang:** Investigation, Data curation; **Tianqing Zhu:** Supervision, Resources.

Author agreement

Dear Editor, We the undersigned declare that this manuscript entitled "PreDyn-IDS: Pretrained Lightweight Open-Set IDS for IoT via Dynamic Confidence and Hierarchical Feature Fusion" is original, has not been published before and is not currently being considered for publication elsewhere. We would like to draw the attention of the Editor to the following publications of one or more of us that refer to aspects of the manuscript presently being submitted. Where relevant copies of such publications are attached. We confirm that the manuscript has been read and approved by all named authors and that there are no other persons who satisfied the criteria for authorship but are not listed. We further confirm that the order of authors listed in the manuscript has been approved by all of us. We understand that the Corresponding Author is the sole contact for the Editorial process. He/she is responsible for communicating with the other authors about progress, submissions of revisions and final approval of proofs. All authors as follows: Xi Luo, GuoHua Yana, LiHua Yin, Gaopan Hou, JingJing Guoc, YanQi Songd, Xiang Lie, WenTing Wangf and TianQing Zhug.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data used in this study are publicly available.

References

- [1] W. Dong, J. Yu, X. Lin, G. Gou, G. Xiong, Deep learning and pre-training technology for encrypted traffic classification: a comprehensive review, *Neurocomputing* 617 (2025) 128444. <https://www.sciencedirect.com/science/article/pii/S0925231224012153>. <https://doi.org/10.1016/j.neucom.2024.128444>
- [2] O. Salman, I.H. Elhajj, A. Kayssi, A. Chehab, A review on machine learning-based approaches for internet traffic classification, *Ann. Telecommun.* (2020) 673–710. <https://doi.org/10.1007/s12243-020-00770-7>
- [3] N. Moustafa, A new distributed architecture for evaluating AI-based security systems at the edge: network TON IoT datasets, *Sustain. Cities Soc.* 70 (2021) 102994. <https://www.sciencedirect.com/science/article/pii/S2210670721002385>. <https://doi.org/10.1016/j.scs.2021.102994>
- [4] R. Nazre, R. Budke, O. Oak, S. Sawant, A. Joshi, A temporal convolutional network-based approach for network intrusion detection, in: 2024 International Conference on Innovative Intelligent Computing and Communication Systems (ICIICS), IEEE, 2024, pp. 1–6. <https://doi.org/10.1109/ICIICS63763.2024.10860234>
- [5] J. Hayes, G. Danezis, K-fingerprinting: a robust scalable website fingerprinting technique, *arXiv: Cryptography and Security* (2015).
- [6] V.F. Taylor, R. Spolaor, M. Conti, I. Martinovic, Robust smartphone app identification via encrypted network traffic analysis, *IEEE Trans. Inf. Forensics Secur.* (2018) 63–78. <https://doi.org/10.1109/tifs.2017.2737970>
- [7] A. Panchenko, F. Lanze, A. Zinnen, M. Henze, J. Pennekamp, K. Wehrle, T. Engel, Website fingerprinting at internet scale, in: Proceedings 2016 Network and Distributed System Security Symposium, 2016. <https://doi.org/10.14722/ndss.2016.23477>
- [8] M. Shen, Y. Liu, L. Zhu, K. Xu, X. Du, N. Guizani, Optimizing feature selection for efficient encrypted traffic classification: a systematic approach, *IEEE Netw.* (2020) 20–27. <https://doi.org/10.1109/mnet.011.1900366>
- [9] T.T.T. Nguyen, G. Armitage, A survey of techniques for internet traffic classification using machine learning, *IEEE Commun. Surv. Tutor.* (2008) 56–76. <https://doi.org/10.1109/surv.2008.080406>
- [10] P. Nimbalkar, D. Kshirsagar, Feature selection for intrusion detection system in internet-of-things (IoT), *ICT Express* 7 (2) (2021) 177–181. <https://www.sciencedirect.com/science/article/pii/S2405959521000588>. <https://doi.org/10.1016/j.icte.2021.04.012>
- [11] V.F. Taylor, R. Spolaor, M. Conti, I. Martinovic, Appscanner: automatic fingerprinting of smartphone apps from encrypted network traffic, in: 2016 IEEE European Symposium on Security and Privacy (EuroS&P), 2016, pp. 439–454. <https://doi.org/10.1109/EuroSP.2016.40>
- [12] T. van Ede, R. Bortolameotti, A. Continella, J. Ren, D.J. Dubois, M. Lindorfer, D.R. Choffnes, M. van Steen, A. Peter, FlowPrint: semi-supervised mobile-app fingerprinting on encrypted network traffic, *Proc. 2020 Netw. Distrib. Syst. Secur. Symp.* (2020). <https://api.semanticscholar.org/CorpusID:211265114>

- [13] C. Liu, Z. Cao, Z. Li, G. Xiong, LaFFT: length-Aware FFT based fingerprinting for encrypted network traffic classification, in: 2018 IEEE Symposium on Computers and Communications (ISCC), 2018, pp. 1–6. <https://doi.org/10.1109/ISCC.2018.8538732>
- [14] H.Y. He, Z. Guo Yang, X.N. Chen, PERT: Payload encoding representation from transformer for encrypted traffic classification, in: 2020 ITU Kaleidoscope: Industry-Driven Digital Transformation (ITU K), 2020. <https://doi.org/10.23919/ituk50268.2020.9303204>
- [15] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, J. Yu, ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification in: The Proceedings of the ACM Web Conference 2022 (2026) pp. 633–642.
- [16] T. Wang, X. Xie, W. Wang, C. Wang, Y. Zhao, Y. Cui, NetMamba: efficient network traffic classification via pre-training unidirectional Mamba in: the 2024 IEEE 32nd International Conference on Network Protocols (ICNP) (2026) 1–11.
- [17] R. Zhao, M. Zhan, X. Deng, Y. Wang, Y. Wang, G. Gui, Z. Xue, Yet another traffic classifier: a masked autoencoder based traffic transformer with multi-level flow representation in: Proceedings of the AAAI Conference on Artificial Intelligence 37 (4) (2023) pp. 5420–5427.
- [18] G. Zhou, X. Guo, Z. Liu, T. Li, Q. Li, K. Xu, Trafficformer: an efficient pre-trained model for traffic data in: 2025 IEEE Symposium on Security and Privacy (SP) (2026) pp. 1844–1860.
- [19] X. Wang, Y. Wang, Y. Lai, Z. Hao, A.X. Liu, Reliable open-set network traffic classification, IEEE Trans. Inf. Forensics Secur. 20 (2025) 2313–2328. <https://doi.org/10.1109/TIFS.2025.3544067>
- [20] X. Yang, F. Tong, F. Jiang, G. Cheng, A lightweight and dynamic open-set intrusion detection for industrial internet of things, IEEE Trans. Inf. Forensics Secur. 20 (2025) 2930–2943.
- [21] S. Yu, R. Zhai, Y. Shen, G. Wu, H. Zhang, S. Yu, S. Shen, Deep Q-network-based open-set intrusion detection solution for industrial internet of things, IEEE Internet Things J. 11 (7) (2023) 12536–12550.
- [22] J. Zhang, Z. Pan, J. Cui, H. Zhong, J. Li, D. He, Toward open-set intrusion detection in VANETs: an efficient meta-recognition approach, IEEE Trans. Netw. Sci. Eng. (2024).
- [23] A. Gu, T. Dao, Mamba: linear-Time sequence modeling with selective state spaces (2023).
- [24] S. Tharewal, M.W. Ashfaq, S.S. Banu, P. Uma, S.M. Hassen, M. Shabaz, Intrusion detection system for industrial internet of things based on deep reinforcement learning, Wirel. Commun. Mob. Comput. 2022 (2022) 9023719. 8 pages. <https://doi.org/10.1155/2022/9023719>
- [25] M. Lotfollahi, R. Zade, M. Siavoshani, M. Saberian, Deep packet: a novel approach for encrypted traffic classification using deep learning, [arXiv:Learning](https://arxiv.org/abs/2017.08.08) (2017).
- [26] C. Liu, L. He, G. Xiong, Z. Cao, Z. Li, FS-Net: A flow sequence network for encrypted traffic classification, in: IEEE INFOCOM 2019 - IEEE Conference on Computer Communications, 2019, pp. 1171–1179. <https://doi.org/10.1109/INFOCOM.2019.8737507>
- [27] J. Tong, Y. Zhang, A real-Time label-Free self-Supervised deep learning intrusion detection for handling new type and few-Shot attacks in IoT networks, IEEE Internet Things J. 11 (19) (2024) 30769–30786. <https://doi.org/10.1109/JIOT.2024.3414492>
- [28] K. He, X. Chen, S. Xie, Y. Li, P. Dollar, R. Girshick, Masked autoencoders are scalable vision learners, in: 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022. <https://doi.org/10.1109/cvpr52688.2022.01553>
- [29] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, Neural Inf. Process. Syst. Inf. Process. Syst. (2017).
- [30] SplitCap [Online], (2026), (Available: <https://www.netresec.com/?page=SplitCap>).
- [31] P. Wang, S. Li, F. Ye, Z. Wang, M. Zhang, PacketCGAN: Exploratory Study of Class Imbalance for Encrypted Traffic Classification Using CGAN, 2019, <https://arxiv.org/abs/1911.12046>. [arXiv:1911.12046](https://arxiv.org/abs/1911.12046) [cs.CR]
- [32] S.E. Finder, R. Amoyal, E. Treister, O. Freifeld, Wavelet Convolutions for Large Receptive Fields, 2024, <https://arxiv.org/abs/2407.05848>. [arXiv:2407.05848](https://arxiv.org/abs/2407.05848) [cs.CV]
- [33] S. Woo, J. Park, J.-Y. Lee, I.S. Kweon, Cbam: convolutional block attention module (2018) 3–19.
- [34] G. Draper-Gil, A.H. Lashkari, M.S.I. Mamun, A.A. Ghorbani, Characterization of encrypted and VPN traffic using time-related features, in: International Conference on Information Systems Security and Privacy, 2016. <https://api.semanticscholar.org/CorpusID:21535780>.
- [35] A.H. Lashkari, G. Draper-Gil, M.S.I. Mamun, A.A. Ghorbani, Characterization of tor traffic using time based features, in: International Conference on Information Systems Security and Privacy, 2017. <https://api.semanticscholar.org/CorpusID:38122314>.